Shared memory parallelism in Julia

Tasks, threads, and `DataFlowTasks.jl`

Luiz M. Faria

ANF Julia4HPC, Fréjus

13 October 2026






Roadmap

- **13:30–14:50 · Multithreading in Julia**
  - Tasks and threads
  - Independent iterations: chunks, buffers, data races, locks, atomics
  - Dependent tasks: `wait`, fork-join, last writers
- **15:10–16:00 · `DataFlowTasks.jl`**
  - Task dependencies inferred from data
  - Under the hood: the Part 1 primitives
  - Tiled LU again, and profiling
- **16:00–17:30 · Hands-on project**
  - Explore the scripts, bring your own code, or a project

Setup: do it now

1. Clone the repository
```
$ git clone https://github.com/maltezfaria/2026-frejus
$ cd 2026-frejus
```
2. Install the packages (`DataFlowTasks.jl`, GraphViz, GLMakie, BenchmarkTools)
```
$ julia --project -e 'using Pkg; Pkg.instantiate()'
```
Precompiling takes a few minutes: let it run while we start
3. Start Julia with threads
```
$ julia --project -t auto
```

Part 1 needs only `Base`: no need to wait for step 2

Table of Contents

Multithreading in Julia
- Tasks and threads
- Independent iterations
- Dependent tasks

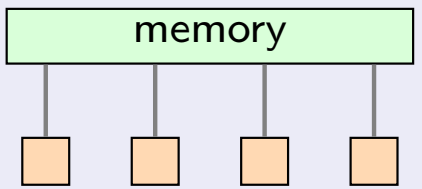
DataFlowTasks.jl

Hands-on

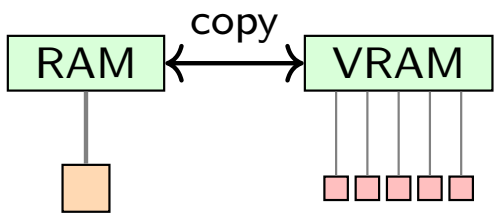
Parallel computing in Julia

Four kinds of concurrent/parallel programming:

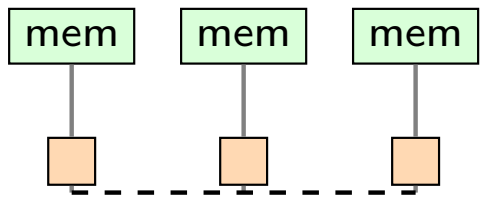
Paradigm	**Memory**	**Julia tools**	**When**
Asynchronous tasks	shared	`Task`, `Channel`, ...	today
Multi-threading	shared	`@threads`, `@spawn`, locks	today
GPU computing	device	`CUDA.jl`, `AMDGPU.jl`, ...	Wednesday
Distributed computing	distributed	`Distributed`, `MPI.jl`, ...	Friday



shared: threads, tasks



device: GPU



distributed: messages

✓ **No explicit communication**

✗ **Synchronization is your job** (data races!)

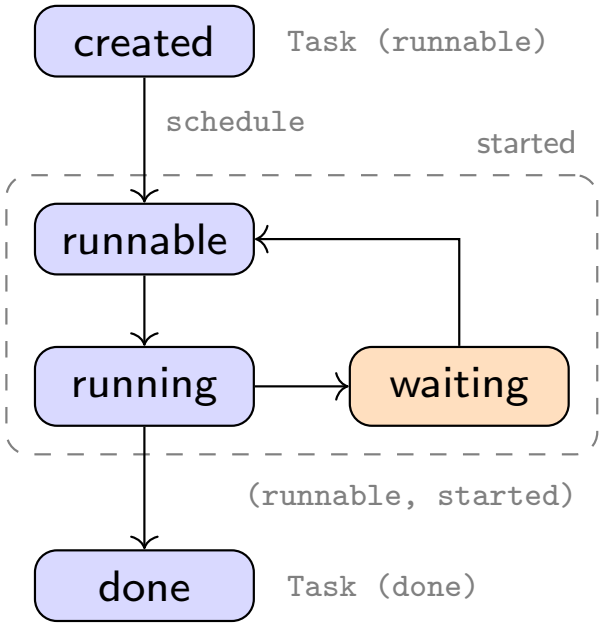
What is a task?

Task: a unit of work that can be suspended and resumed

a.k.a. coroutine, lightweight thread

```
f = () -> (sleep(1); 1 + 1)

t = Task(f)   # create
schedule(t)   # start: returns at once
fetch(t)      # wait 1 s, get result
# 2
```



Syntactic sugar:

```
t = @task f()    # Task(f)
t = @async f()   # Task + schedule: current thread (discouraged)
t = @spawn f()   # Task + t.sticky = false + schedule: any thread
```

>_ Live demo: REPL

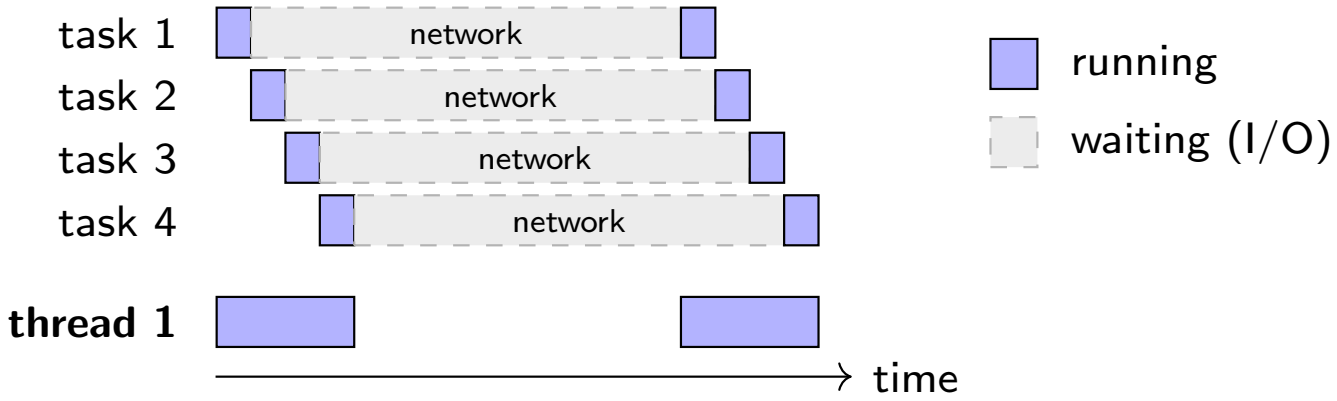
Example: concurrency

I/O-bound: 20 queries to HAL, single thread (`julia -t 1`)

```
using Downloads: download
api = "https://api.archives-ouvertes.fr/search"
# HAL publications of a given year
hal(year) = "$api/?q=producedDateY_i:$year"
urls = hal.(2005:2024)

@time map(download, urls)                                  # 2.8 s
@time fetch.(map(u -> @spawn(download(u)), urls))          # 0.2 s
```

Waiting for the network suspends the task ⇒ the thread runs another one



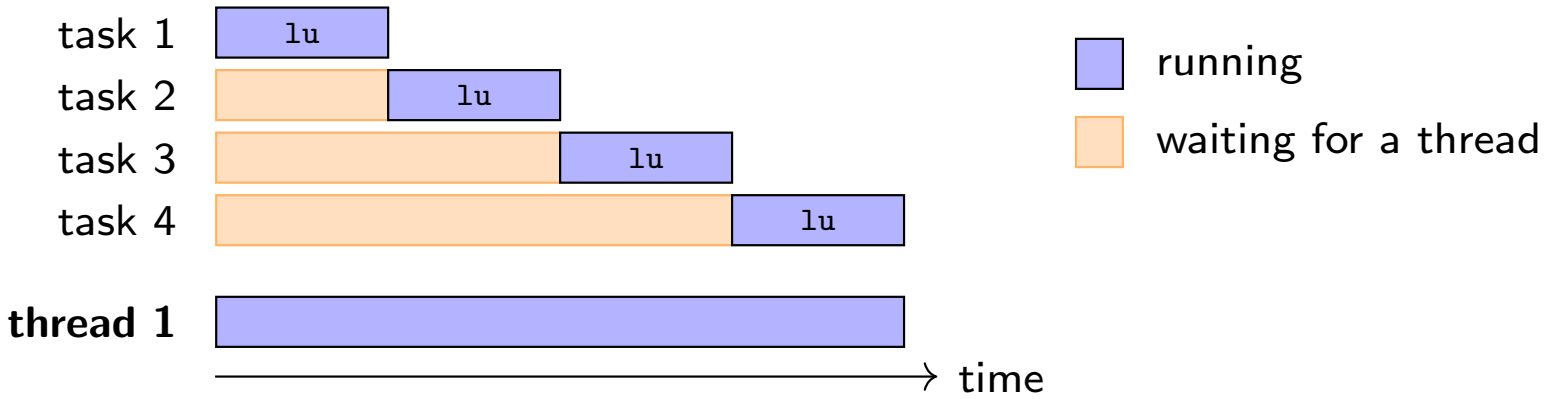
</> Live demo: `concurrency.jl`

Example: no concurrency gain

CPU-bound: factor 10 matrices, single thread (`julia -t 1`)

```
using LinearAlgebra
BLAS.set_num_threads(1)  # more on this later
As = [rand(2000, 2000) for _ in 1:10]

@time map(lu, As)                                  # 1.1 s
@time fetch.(map(A -> @spawn(lu(A)), As))  # 1.1 s
```



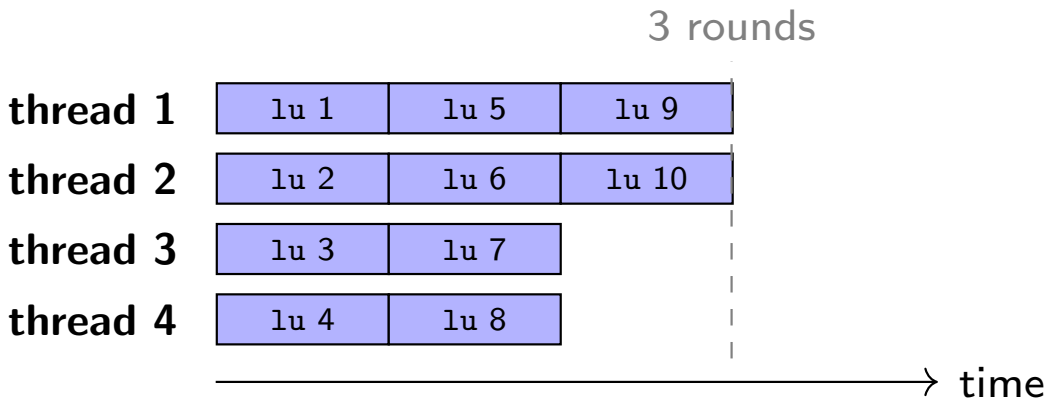
- Cooperative: a task gives its thread back only when it **waits** or **finishes**, and `lu` never waits

</> Live demo: `parallelism.jl`

Same code, more threads

CPU-bound: factor 10 matrices, four threads (`julia -t 4`)

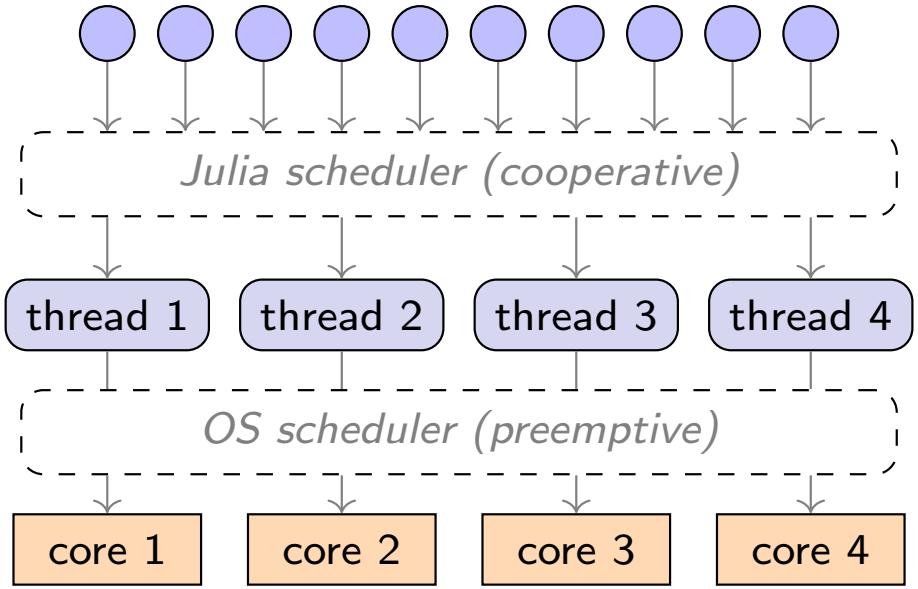
```
@time map(lu, As)                                  # 1.1 s
@time fetch.(map(A -> @spawn(lu(A)), As))          # 0.4 s
```



- Concurrency: many tasks *in progress* (one thread is enough)
- Parallelism: many tasks *running at once* (needs threads)
- 10 tasks on 4 threads: 3 rounds $\Rightarrow$ about $3\times$, not $4\times$

</> Live demo: `parallelism.jl`

Tasks, threads, and cores



Tasks: unit of work
many, cheap: `@spawn`

Threads: unit of execution
fixed number: `julia -t N`
`-t 4` = 4 default + 1 interactive
`nthreads()`: default pool

Cores: unit of hardware
`Sys.CPU_THREADS`

A task can move between threads, e.g. when it resumes after a `wait`
⇒ `threadid()` can change during a task

You write tasks, not threads

Table of Contents

Multithreading in Julia
- Tasks and threads
- Independent iterations
- Dependent tasks

DataFlowTasks.jl

Hands-on

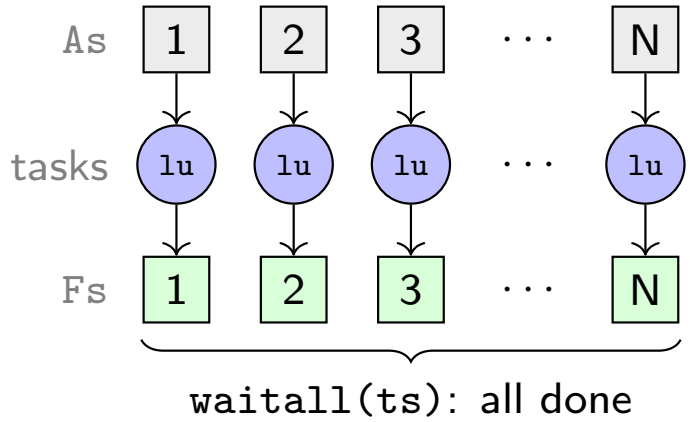
Independent iterations

A common pattern: N iterations that do not depend on each other

iteration i reads its own inputs, writes its own output (solves, sweeps, ...)

```
As = [rand(2000, 2000) for _ in 1:N]
Fs = Vector{Any}(undef, N)
```

```
ts = Task[]
for i in 1:N
    t = @spawn Fs[i] = lu(As[i])
    push!(ts, t)
end
waitall(ts)
```



`@spawn ex` = `Task(() -> ex)` + `sticky = false` + `schedule`

- Each task writes its own slot `Fs[i]` ⇒ no coordination
- The loop returns at once: wait before reading `Fs`
- `@sync for ... @spawn ... end`: waits for all its `@spawns`

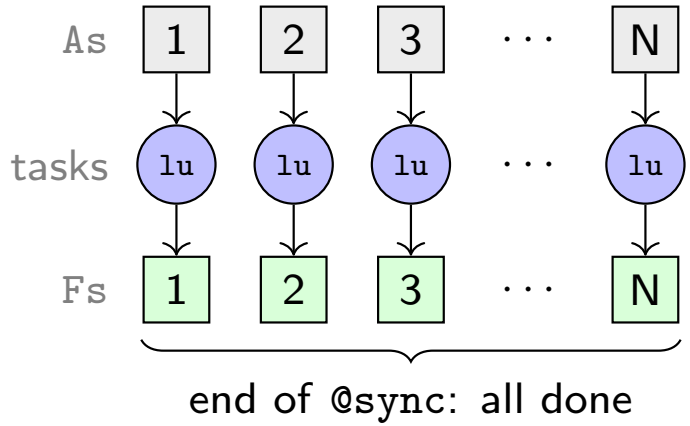
</> Live demo: `threads_loop.jl`

Independent iterations

A common pattern: N iterations that do not depend on each other

iteration i reads its own inputs, writes its own output (solves, sweeps, ...)

```
As = [rand(2000, 2000) for _ in 1:N]
Fs = Vector{Any}(undef, N)
@sync for i in 1:N
    @spawn Fs[i] = lu(As[i])
end
```



`@spawn ex` = `Task(() -> ex)` + `sticky = false` + `schedule`

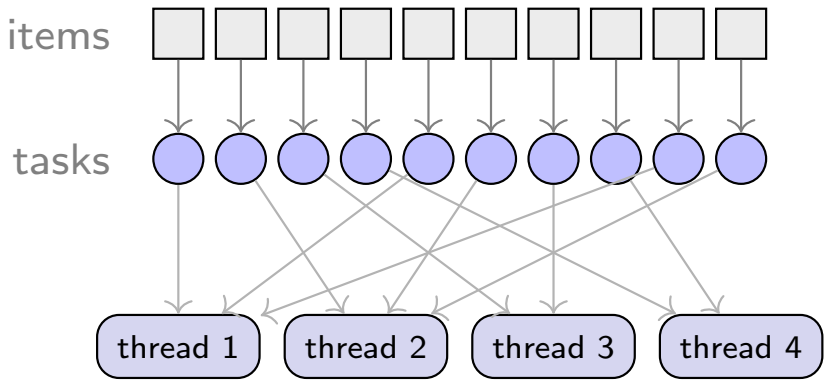
- Each task writes its own slot `Fs[i]` ⇒ no coordination
- The loop returns at once: wait before reading `Fs`
- `@sync for ... @spawn ... end`: waits for all its `@spawns`

</> Live demo: `threads_loop.jl`

Chunking: items → tasks → threads

One task per item

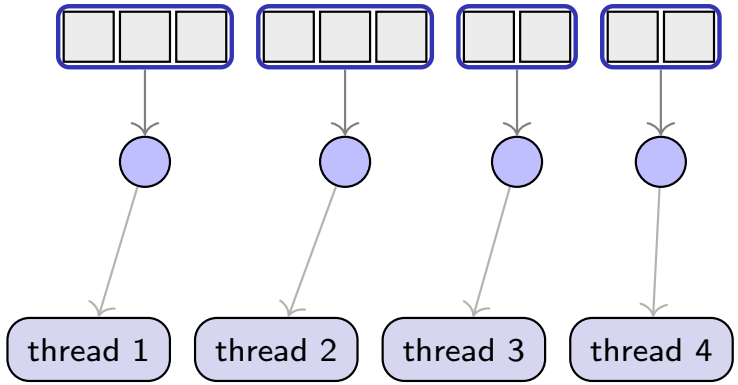
`@spawn` in the loop



10 tasks: best balance, most overhead

One task per chunk

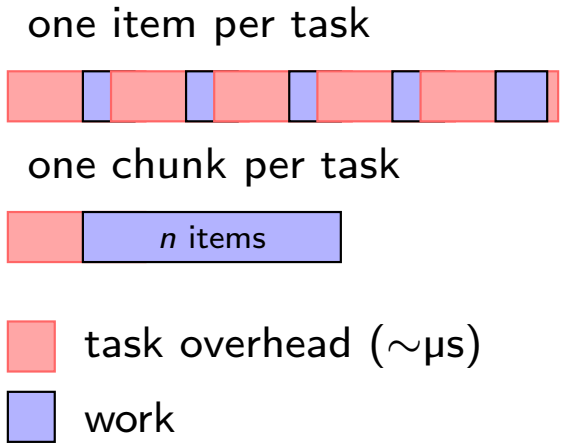
`@spawn` per chunk



4 tasks: least overhead, balance at risk

Chunks in code: granularity

```
n = cld(N, nchunks)  # chunk size
@sync for idxs in partition(1:N, n)
    @spawn for i in idxs
        ys[i] = f(xs[i])
    end
end
```



- Every task pays a fixed cost: create, schedule, wait (∼µs)
- A chunk of n items pays it once: keep $n\times$ (cost per item) $\gg$ task cost
- Too few chunks: one slow chunk leaves threads idle $\Rightarrow$ balance (next)

⚠ Cost per item: usually unknown, and varies from run to run
$\Rightarrow$ hard to predict: measure, and keep a few chunks per thread

</> Live demo: `granularity.jl`

`@threads`: a convenience macro

```
@threads for i in 1:N
    ys[i] = f(xs[i])
end
```

= one task per chunk, then wait for all: the code of the previous slide

Schedules: `@threads :greedy for i in ...`

`:dynamic`	default: ∼`nthreads()` contiguous chunks
`:greedy`	`nthreads()` tasks, each takes the next item when free: uneven work, any iterator
`:static`	one task pinned per thread: `threadid()` constant; cannot be nested, discouraged

Use it for independent loops. To combine results (e.g. a sum): protect the shared state (lock, atomic) *later*

Live demo: `threads_loop.jl`

Reusing memory: buffers

Many systems: $N = 100$ solves of size 1000, single thread

Allocate every time:

```
for i in 1:N
    F = lu(As[i])
    xs[i] = F \ bs[i]
end
```

Buffer: allocate once, overwrite:

```
F = lu(As[1])
for i in 1:N
    lu!(F, As[i])  # writes into F
    xs[i] = F \ bs[i]
end
```

Memory: 766 MiB allocated in total vs. 8 MiB, one `F` reused

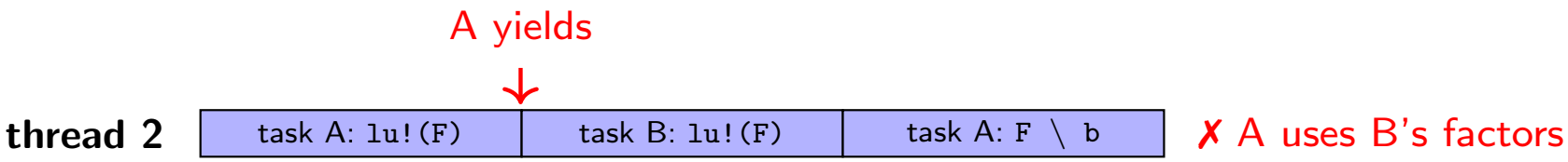
With threads: one buffer per what?

</> Optional demo: `buffers.jl`

Tempting: one buffer per thread

```
Fs = [lu(As[1]) for _ in 1:maxthreadid()]  # one per thread id
@threads for i in 1:N
    F = Fs[threadid()]
    lu!(F, As[i])
    xs[i] = F \ bs[i]
end
```

- Runs, right answer. . . and wrong



- At a yield, a task may migrate, or another task may take its thread
- No yield in `lu!` today ⇒ correct by luck, until it is not

You write tasks, not threads

</> Optional demo: `threadid.jl`

Buffers belong to tasks

By hand: one buffer per chunk task

```
n = cld(N, nthreads())  # chunk size
@sync for idxs in partition(1:N, n)
    @spawn begin
        F = lu(As[1])  # buffer
        for i in idxs
            lu!(F, As[i])
            xs[i] = F \ bs[i]
        end
    end
end
```

With `@threads`: `OncePerTask`

```
F0 = lu(As[1])
buf = OncePerTask(() -> copy(F0))
@threads for i in 1:N
    F = buf()  # buffer
    lu!(F, As[i])
    xs[i] = F \ bs[i]
end
```

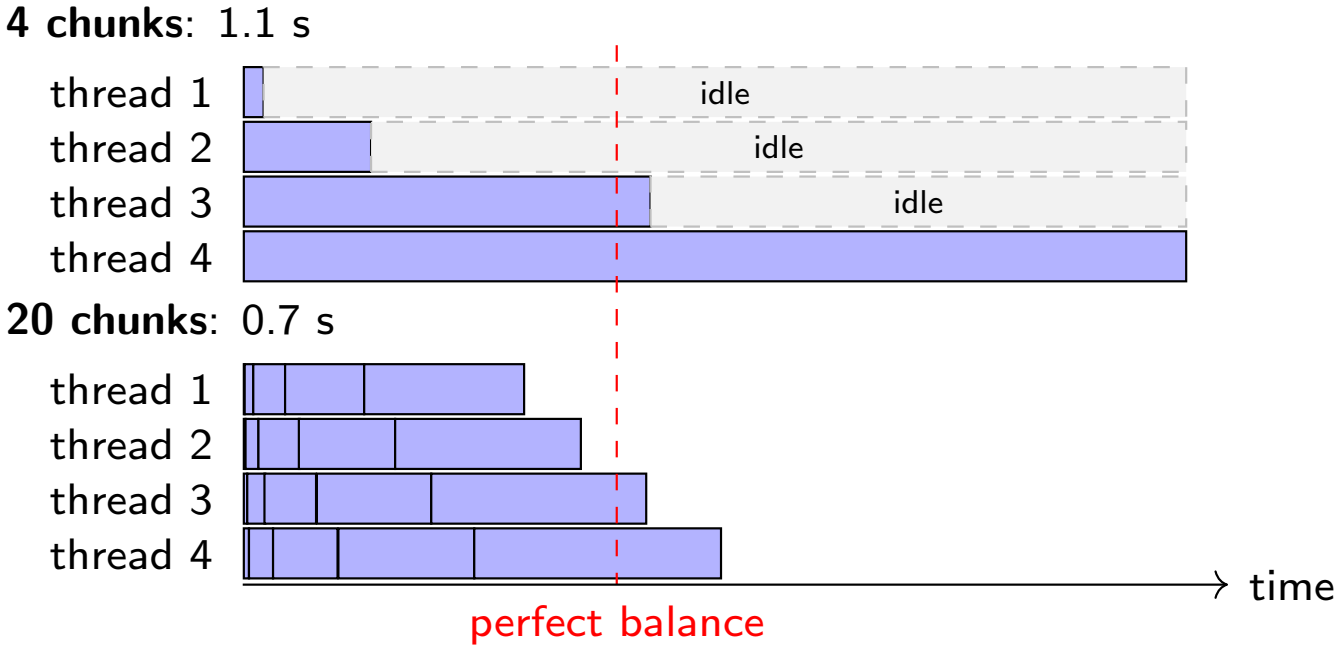
Julia $\geq$ 1.12

- Buffers = tasks = chunks $\approx$ threads, not iterations: 20$\times$ less memory
- `buf()`: runs the initializer once per task, then returns the same buffer
- Each loop: new tasks $\Rightarrow$ new buffers

</> Live demo: `buffers_threads.jl`

Load balancing: how many chunks?

Unequal work: sizes 200 $\rightarrow$ 1600, cost $\propto m^3$: the last items dominate

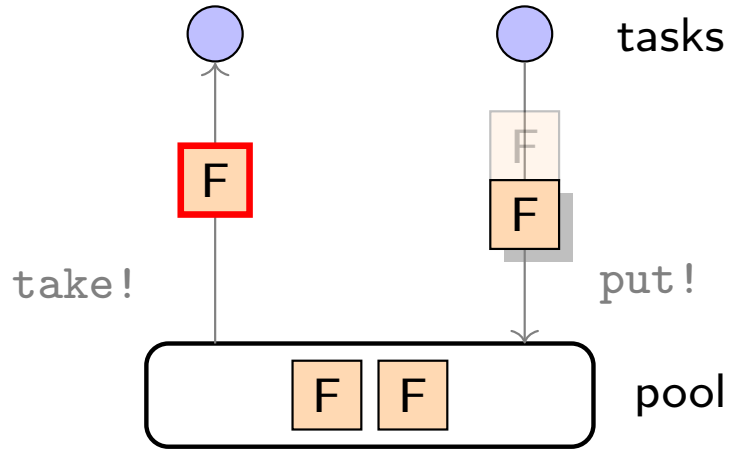


- 4 chunks: the last one holds 63% of the work
- More chunks: a free thread takes the next one $\Rightarrow$ balance
- Cost: more tasks, more buffers

</> Optional demo: `load_balance.jl`

Borrowing buffers: a pool

```
F0 = lu(As[1])
pool = Channel{typeof(F0)}(nthreads())
for _ in 1:nthreads()
    put!(pool, copy(F0))  # cheap
end
@threads for i in 1:N
    F = take!(pool)  # wait if empty
    lu!(F, As[i])
    xs[i] = F \ bs[i]
    put!(pool, F)    # give it back
end
```

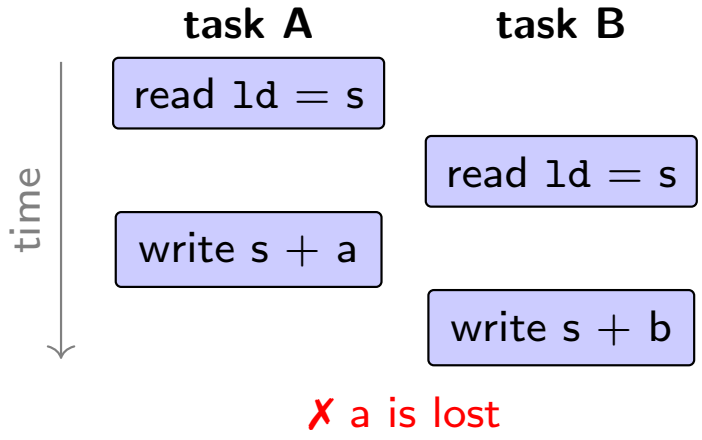


- Tasks borrow a buffer, do not own one: buffers outlive the loop
- Any schedule, any number of tasks: bounded number of buffers
- Setup runs on one thread: one `lu`, then cheap copies

Live demo: `buffers_threads.jl`

Accumulating: sum of log-determinants

```
ld = 0.0
@threads for i in 1:N
    F = lu(As[i])
    xs[i] = F \ bs[i]
    ld += logdet(F)   # race!
end
```



`-t 1` exact ✓

`-t 4` 3–8 % too small, varies ✗

- `+=` is read, add, write: tasks interleave ⇒ lost updates
- Wrong silently: no error, plausible number

</> Live demo: `accumulate.jl`

Fixing the race

Combine the `d = logdet(F)` of each iteration:

Lock

one task at a time; others queue

```
ld = Lockable(Ref(0.0))
@lock ld ld[][] += d  # in the loop
```

Atomic

indivisible update; primitive types only

```
ld = Atomic{Float64}(0.0)
atomic_add!(ld, d)    # in the loop
```

No sharing

each task returns a partial sum; own tasks

```
ts = map(chunks) do idxs
    @spawn sum(i -> logdet(lu(As[i])), idxs)
end
ld = sum(fetch.(ts))
```

- Lock, atomic: order of additions varies ⇒ last digits vary

</> Live demo: `accumulate.jl`

Fixing the race: a lock

```
ld = Lockable(Ref(0.0))
@threads for i in 1:N
    F = lu(As[i])
    xs[i] = F \ bs[i]
    d = logdet(F)  # outside the lock
    @lock ld ld[][] += d
end
@lock ld ld[][]
```

`@lock l expr` is sugar for:

```
lock(l)
try
    expr
finally
    unlock(l)
end
```

- `lock`: one task at a time in the critical section; others wait
- `Lockable`: value + lock, value only reached under the lock
- Keep the critical section short: compute outside, update inside
- Correct, same speed: 0.49 s (`-t 4`)

</> Live demo: `accumulate.jl`

Table of Contents

Multithreading in Julia
- Tasks and threads
- Independent iterations
- Dependent tasks

DataFlowTasks.jl

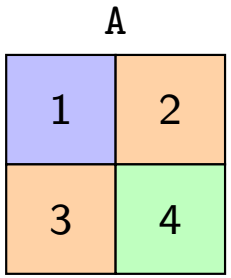
Hands-on

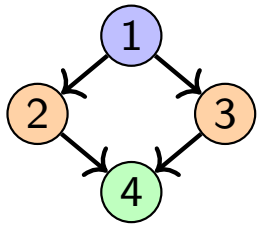
Example: 2×2 block LU

One big matrix instead of many: tiles $\Rightarrow$ tasks depend on each other

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ & U_{22} \end{pmatrix}$$

1	$A_{11} = L_{11} U_{11}$	`lu!`
2	$U_{12} = L_{11}^{-1} A_{12}$	`ldiv!`
3	$L_{21} = A_{21} U_{11}^{-1}$	`rdiv!`
4	$L_{22} U_{22} = A_{22} - L_{21} U_{12}$	`mul!`, `lu!`





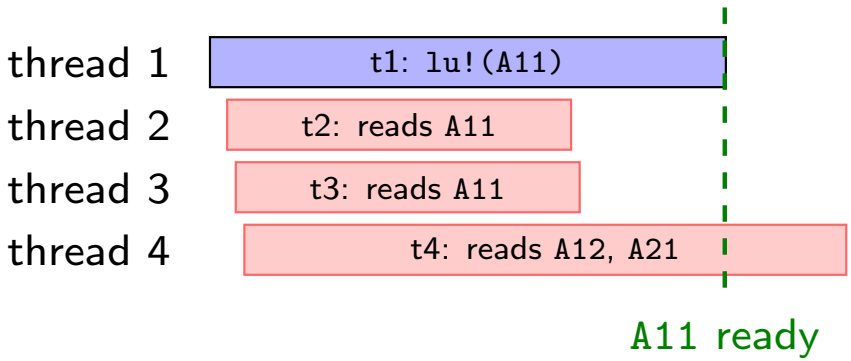
- In place: L and U overwrite A, tile by tile
- Order matters: 2 and 3 need 1; 4 needs 2 and 3

⚠ Dependent tasks: in general much harder than independent iterations
$\Rightarrow$ who waits for whom, and when?

Coordination is your job

One task per step, no coordination:

```
t1 = @spawn lu!(A11, NoPivot())
t2 = @spawn ldiv!(UnitLowerTriangular(A11), A12)  # needs t1!
t3 = @spawn rdiv!(A21, UpperTriangular(A11))      # needs t1!
t4 = @spawn begin                                 # needs t2, t3!
    mul!(A22, A21, A12, -1, 1)
    lu!(A22, NoPivot())
end
```



✗ read before they are ready

- `-t 1`: error 5×10^{-16}, correct by luck (tasks run in spawn order)
- `-t 4`: error $10^{7} \ldots 10^{11}$, changes every run: silent, nondeterministic

</> Live demo: `lu_nowait.jl`

Coordinating tasks

- **Protect shared data:** `lock`, atomics ✓ seen
- **Communicate:** `Channel` ✓ seen
- **Order:** `wait`, `fetch` now

> `wait(x)`: suspend until `x` says go

every blocking call is a `wait` underneath: `lock`, `take!`, `fetch`, `@sync`, `sleep`

`x`	**wait until...**
`Task`	it is done
`Condition`	`notify(x)`
`Event`	`notify(x)`
`Channel`	data is available
`Timer`	it fires

`fetch` $\approx$ `wait` + read result

```
function fetch(t::Task)
    wait(t)
    return t.result
end
```

simplified from Julia's `Base`

Coordinating tasks

- **Protect shared data:** `lock`, atomics ✓ seen
- **Communicate:** `Channel` ✓ seen
- **Order:** `wait`, `fetch` now

`wait(x)`: suspend until `x` says go

every blocking call is a `wait` underneath: `lock`, `take!`, `fetch`, `@sync`, `sleep`

`x`	**wait until...**
`Task`	it is done
`Condition`	`notify(x)`
`Event`	`notify(x)`
`Channel`	data is available
`Timer`	it fires

`sleep` ≈ `wait` on a timer

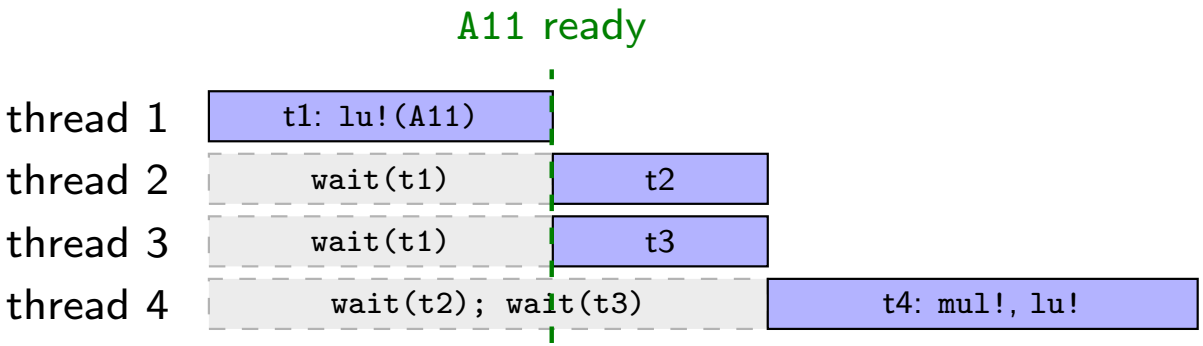
```
function sleep(s)
    wait(Timer(s))
end
```

simplified from Julia's `Base`

Dependencies by hand: `wait`

Each task waits for its inputs:

```
t1 = @spawn lu!(A11, NoPivot())
t2 = @spawn (wait(t1); ldiv!(UnitLowerTriangular(A11), A12))
t3 = @spawn (wait(t1); rdiv!(A21, UpperTriangular(A11)))
t4 = @spawn begin
    wait(t2); wait(t3)
    mul!(A22, A21, A12, -1, 1)
    lu!(A22, NoPivot())
end
wait(t4)
```

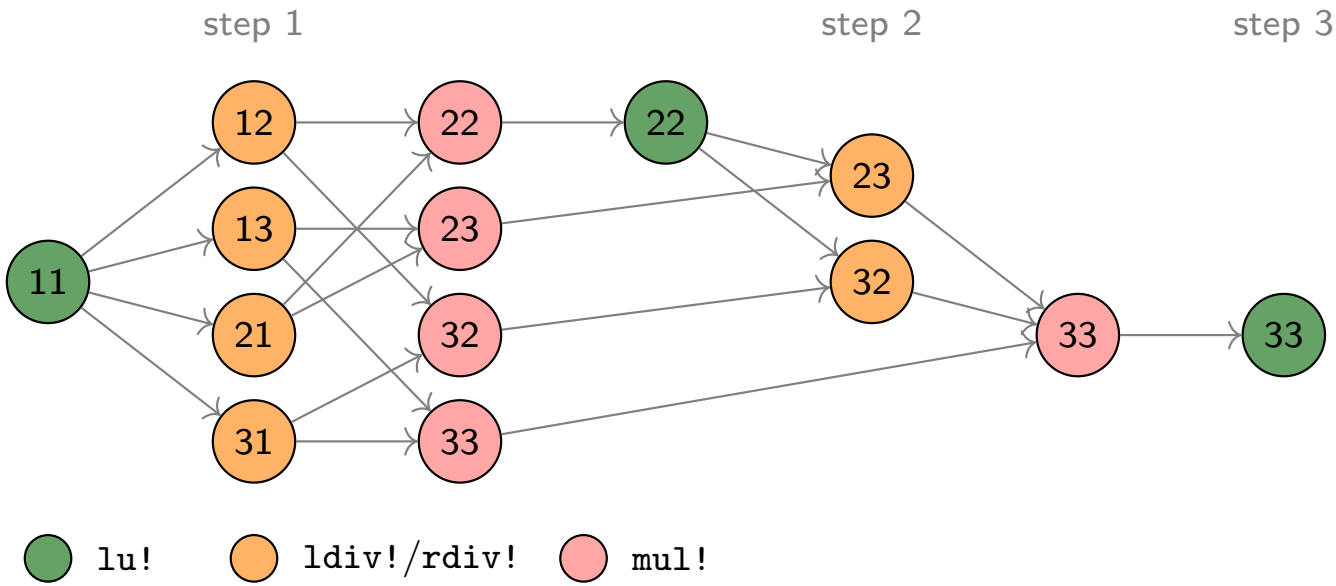


- Correct on any number of threads; one `wait` per edge of the graph
- Only 2 $\|$ 3; the two `lu!` run in sequence: 2.8 s $\to$ 2.5 s $\Rightarrow$ need more tiles

</> Live demo: `lu_2x2.jl`

More tiles, more dependencies

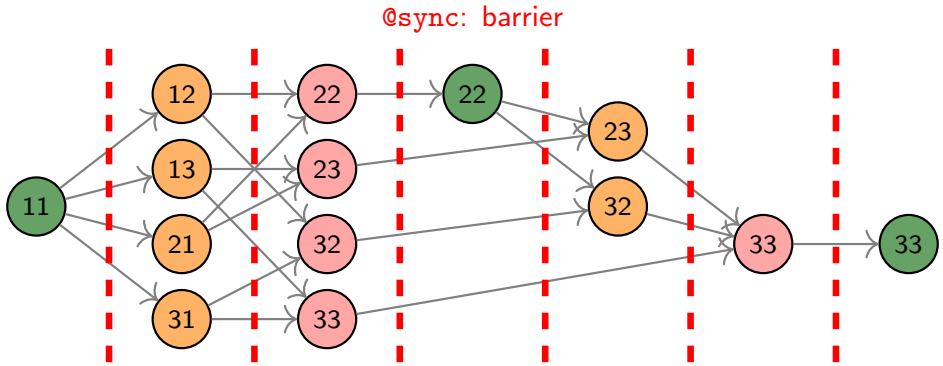
3×3 tiles: one node = one kernel on one tile, one arrow = one `wait`



- 3×3: 14 tasks, 21 edges; $n \times n$: $\sim n^3/3$ tasks
- By hand: track who last wrote each tile: error-prone
- How to enforce them? Two ways by hand: phases, or last writers

Fork-join: dependencies as phases

```
for k in 1:n
    lu!(T[k,k], NoPivot())
    @sync begin  # panels
        for j in k+1:n; @spawn ldiv!(...); end
        for i in k+1:n; @spawn rdiv!(...); end
    end
    @sync for j in k+1:n, i in k+1:n  # updates
        @spawn mul!(T[i,j], T[i,k], T[k,j], -1, 1)
    end
end
```



- Parallel within a phase; wait for all of it before the next
- Simple, often good (0.86 → 0.18 s); but barriers idle threads

</> Live demo: `lu_tiled.jl`

Tasks by hand: a last-writer table

No barriers. Keep `W[i,j]`: the last task that wrote tile (i, j)

Each new task: **1.** waits for the last writers of the tiles it uses; **2.** becomes the last writer of the tile it writes

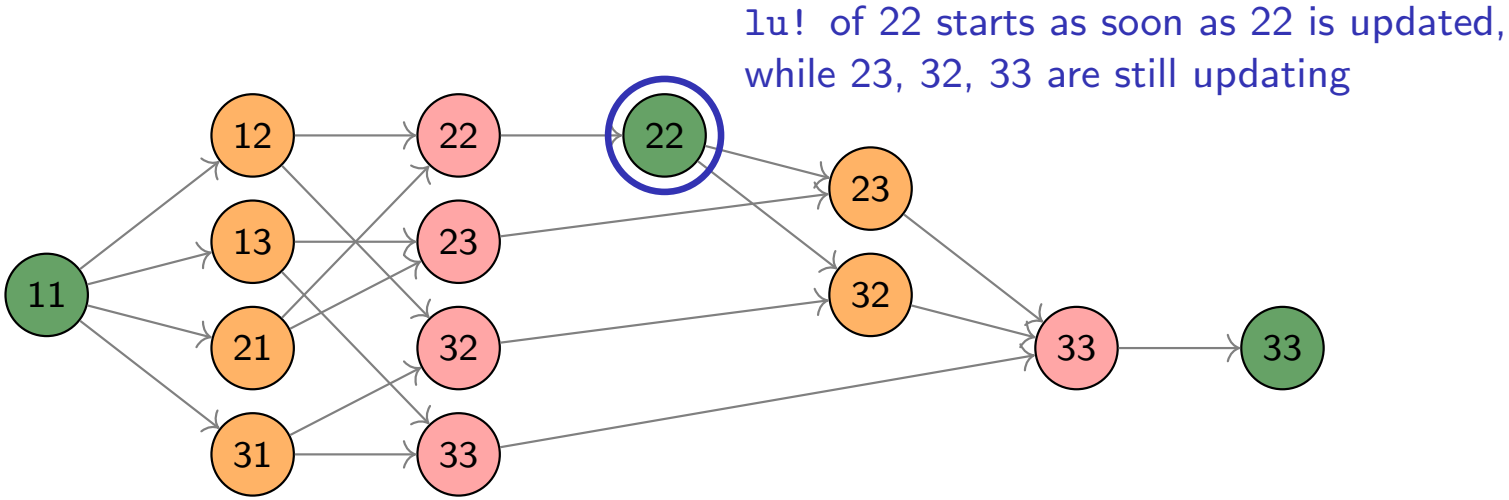
`W` after step 1

`lu!` 11	`ldiv!` 12	`ldiv!` 13
`rdiv!` 21	`mul!` 22	`mul!` 23
`rdiv!` 31	`mul!` 32	`mul!` 33

Step 2, in loop order:

- `lu!` of 22 waits for `W[2,2]` (`mul!` 22) then `W[2,2]` = this task
- `ldiv!` of 23 waits for `W[2,2]`, `W[2,3]`
- `mul!` of 33 waits for `W[3,2]`, `W[2,3]`, `W[3,3]`

Tasks by hand: no barriers



- The waits are the arrows, found while running the sequential loop
- A task starts when its inputs are ready: step $k+1$ overlaps step k
- Fork-join waits for the whole step instead

Tasks by hand: the code

```
spawn_after(f, deps...) = @spawn (waitall(deps); f())
W = fill(@spawn(nothing), n, n)  # last writers
for k in 1:n
    W[k,k] = spawn_after(W[k,k]) do
        lu!(T[k,k], NoPivot())
    end
    for j in k+1:n  # panels
        W[k,j] = spawn_after(W[k,k], W[k,j]) do
            ldiv!(UnitLowerTriangular(T[k,k]), T[k,j])
        end
        W[j,k] = spawn_after(W[k,k], W[j,k]) do
            rdiv!(T[j,k], UpperTriangular(T[k,k]))
        end
    end
    for j in k+1:n, i in k+1:n  # updates
        W[i,j] = spawn_after(W[i,k], W[k,j], W[i,j]) do
            mul!(T[i,j], T[i,k], T[k,j], -1, 1)
        end
    end
end
waitall(W)
```

- The sequential loop, each kernel wrapped in `spawn_after`
- deps: evaluated now, when the task is created
- 8 threads: fork-join 0.18 s, tasks 0.16 s
- Correct only because a tile that is read is never written again

</> Live demo: `lu_tiled.jl`

Table of Contents

Multithreading in Julia
- Tasks and threads
- Independent iterations
- Dependent tasks

`DataFlowTasks.jl`

Hands-on

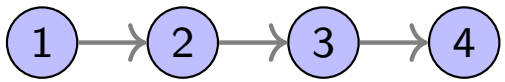
Overview

- `DataFlowTasks.jl` is a Julia package dedicated to parallel programming on **multi-core shared memory CPUs**
- `Task` dependencies inferred from data annotations: `@R`, `@W`, `@RW`
- Simple API: the `@dspawn` macro

In the spirit of StarPU, PaRSEC (not in ambition!)

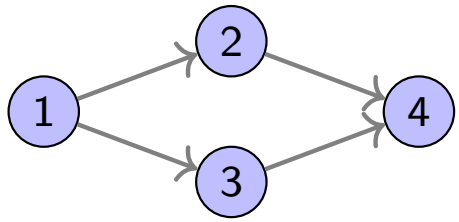
Sequential

```
function foo!(A, b)
  fill!(b, 0)
  view(b, 1:2) .+= 2
  view(b, 3:4) .+= 3
  A \ b
end
```



With `DataFlowTasks.jl`: one task per line

```
function foo!(A, b)
  @dspawn fill!(@W(b), 0)           # task 1
  @dspawn @RW(view(b, 1:2)) .+= 2 # task 2
  @dspawn @RW(view(b, 3:4)) .+= 3 # task 3
  @dspawn @R(A) \ @R(b)             # task 4
end
```

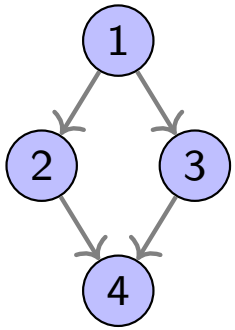


Task based parallelism

- A task is a unit of execution or a unit of work
- `Task` objects can be created using `@task`
- Once created, `Task` objects must be scheduled for execution
- Usually, `Tasks` are created + scheduled using `@spawn`
- Responsibility of synchronizing tasks is left to the programmer

Sequential

```
                          # task
b = zeros(4)              # 1: initialization
b[1:2] .+= 2              # 2: first half
b[3:4] .+= 3              # 3: second half
res = A \ b               # 4: entire vector
```

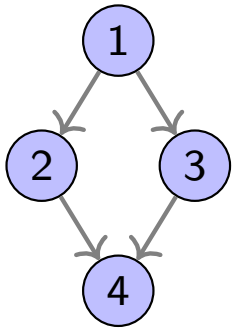


Task based parallelism

- A task is a unit of execution or a unit of work
- `Task` objects can be created using `@task`
- Once created, `Task` objects must be scheduled for execution
- Usually, `Tasks` are created + scheduled using `@spawn`
- Responsibility of synchronizing tasks is left to the programmer

Synchronizing tasks with return values

```
t1 = @spawn zeros(4)                              # 1
t2 = @spawn (b = fetch(t1); b[1:2] .+ 2)          # 2
t3 = @spawn (b = fetch(t1); b[3:4] .+ 3)          # 3
t4 = @spawn A \ vcat(fetch(t2), fetch(t3))        # 4
fetch(t4) # get result
```

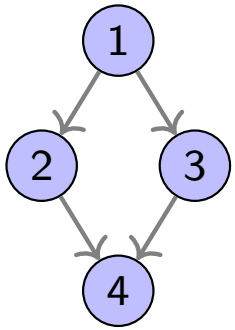


Task based parallelism

- A task is a unit of execution or a unit of work
- `Task` objects can be created using `@task`
- Once created, `Task` objects must be scheduled for execution
- Usually, `Tasks` are created + scheduled using `@spawn`
- Responsibility of synchronizing tasks is left to the programmer

Synchronizing tasks with explicit waits

```
b = Vector{Float64}(undef, 4)
t1 = @spawn fill!(b, 0)                          # 1
t2 = @spawn (wait(t1); view(b,1:2) .+= 2)        # 2
t3 = @spawn (wait(t1); view(b,3:4) .+= 3)        # 3
t4 = @spawn (wait(t2); wait(t3); A \ b)          # 4
fetch(t4) # get result
```



Motivation for DataFlowTasks

- Reasoning about Task interdependencies can be challenging
- Especially for algorithms making constant re-use of data
- Sometimes, it is simpler to reason about how Tasks depend on data than how Tasks depend on each other

Synchronizing tasks

```
b = rand(4)
t1 = @spawn fill!(b, 0)                          # W  access to b
t2 = @spawn (wait(t1); view(b,1:2) .+= 2)        # RW access to b[1:2]
t3 = @spawn (wait(t1); view(b,3:4) .+= 3)        # RW access to b[3:4]
t4 = @spawn (wait(t2); wait(t3); A \ b)          # R  access to A and b
fetch(t4)
```

Motivation for `DataFlowTasks`

- Reasoning about `Task` interdependencies can be challenging
- Especially for algorithms making constant re-use of data
- Sometimes, it is simpler to reason about how `Tasks` depend on data than how `Tasks` depend on each other

Synchronizing tasks

```
b = rand(4)
t1 = @spawn fill!(b, 0)                          # W  access to b
t2 = @spawn (wait(t1); view(b,1:2) .+= 2)        # RW access to b[1:2]
t3 = @spawn (wait(t1); view(b,3:4) .+= 3)        # RW access to b[3:4]
t4 = @spawn (wait(t2); wait(t3); A \ b)          # R  access to A and b
fetch(t4)
```

Would like to declare only the task-to-data dependencies

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



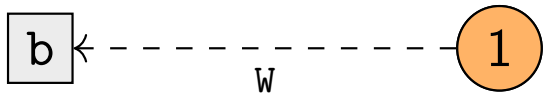
new task compared with data access dependency

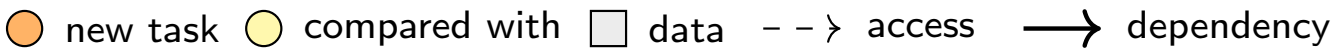
User-written code

```
b = rand(4)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



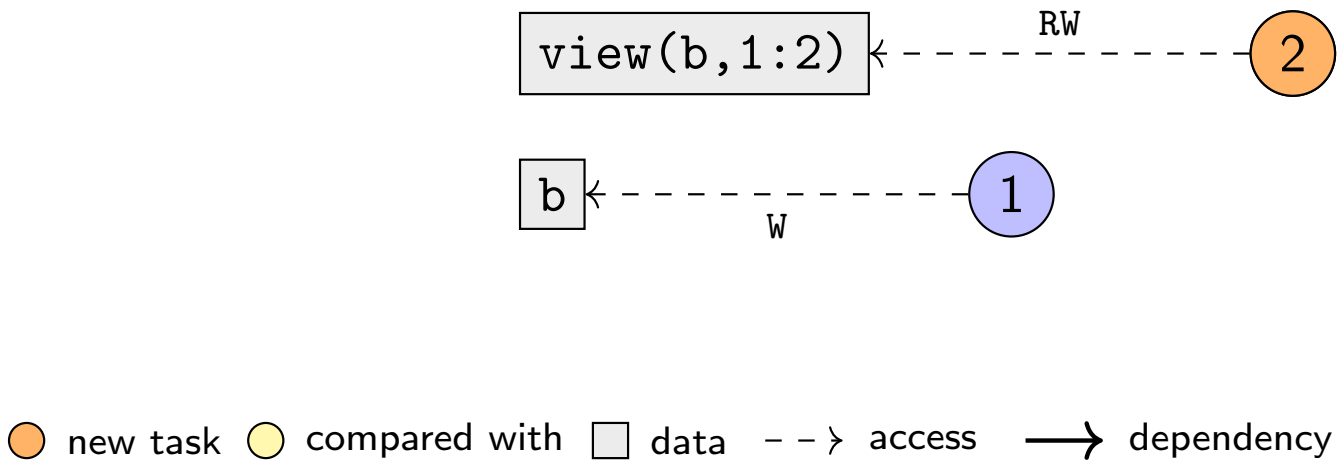


User-written code

```
b = rand(4)
@dspawn fill!(@W(b),0)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies

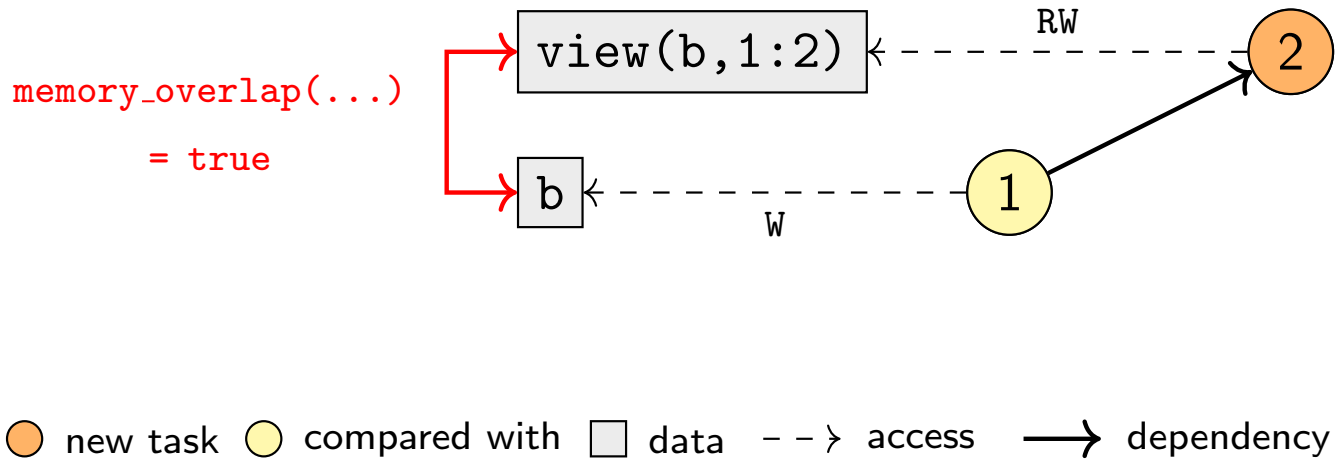


User-written code

```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

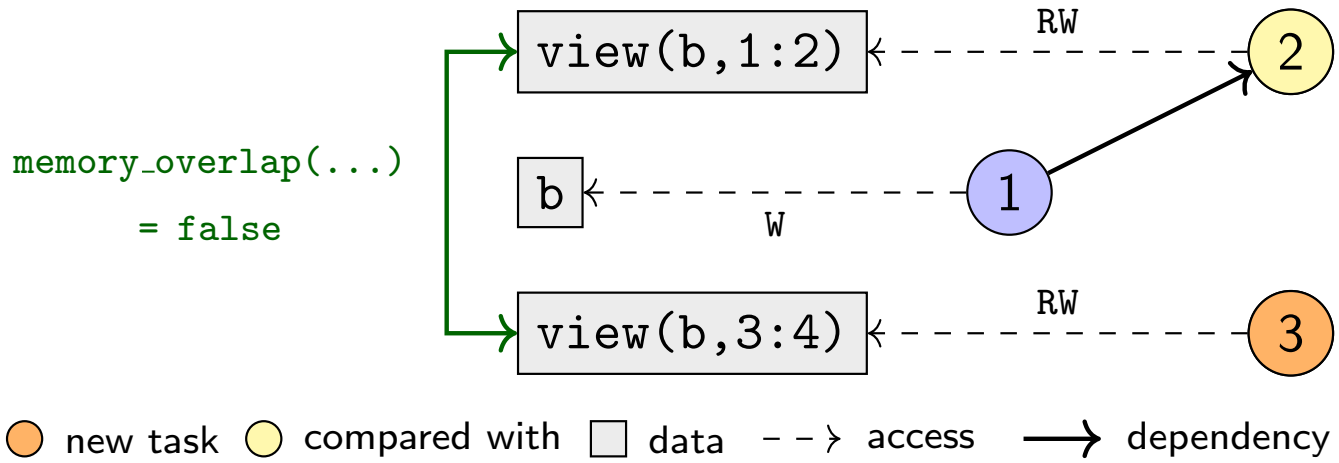
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

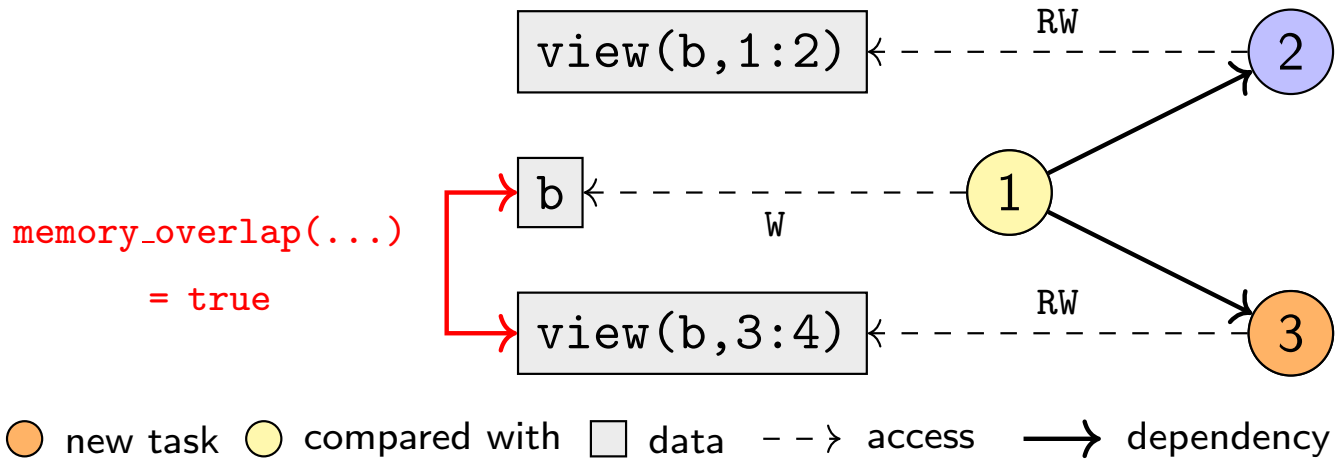
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

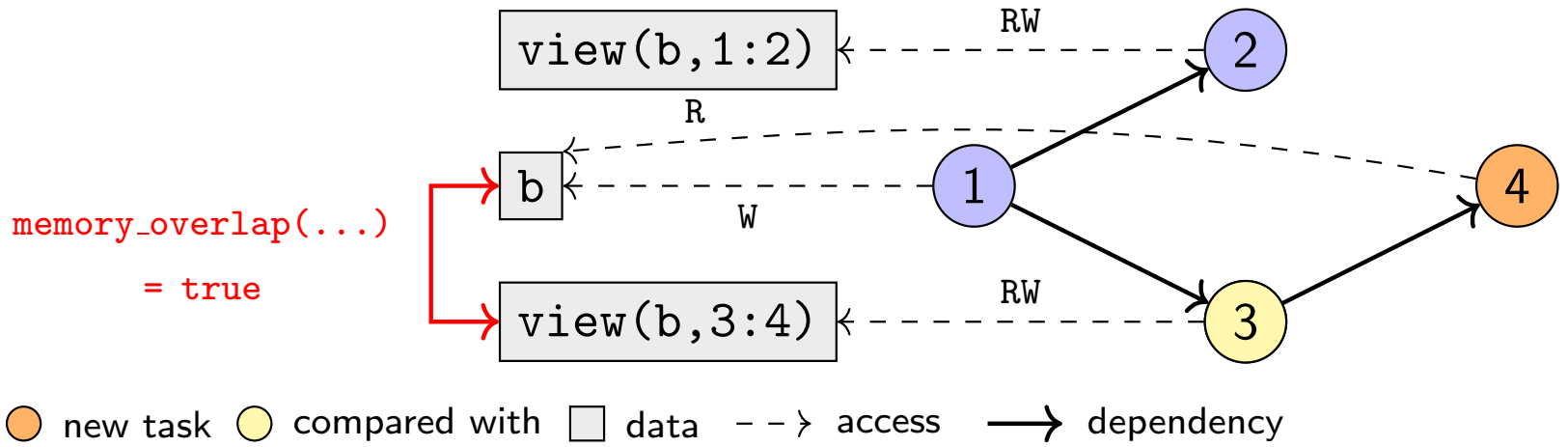
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

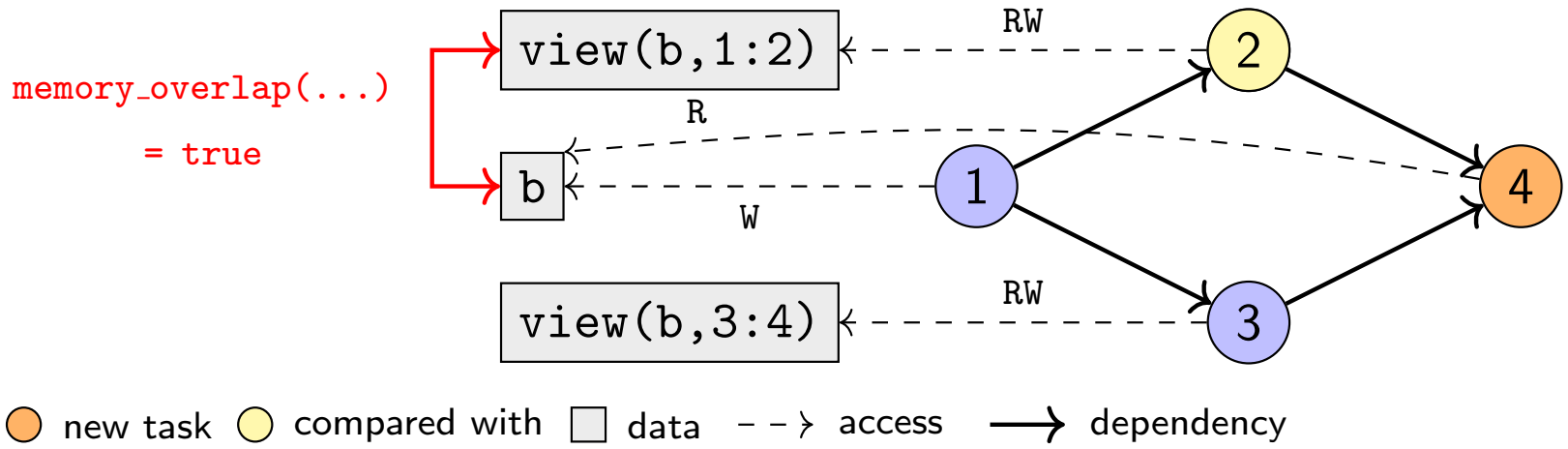
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
t4 = @dspawn A \ @R(b)
fetch(t4)
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

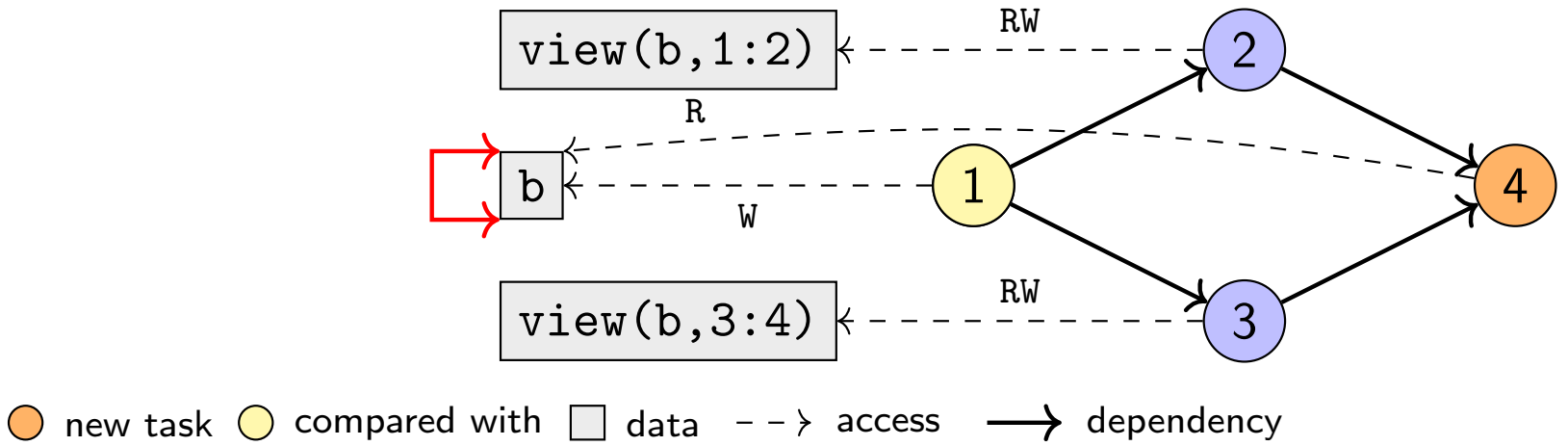
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
t4 = @dspawn A \ @R(b)
fetch(t4)
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



User-written code

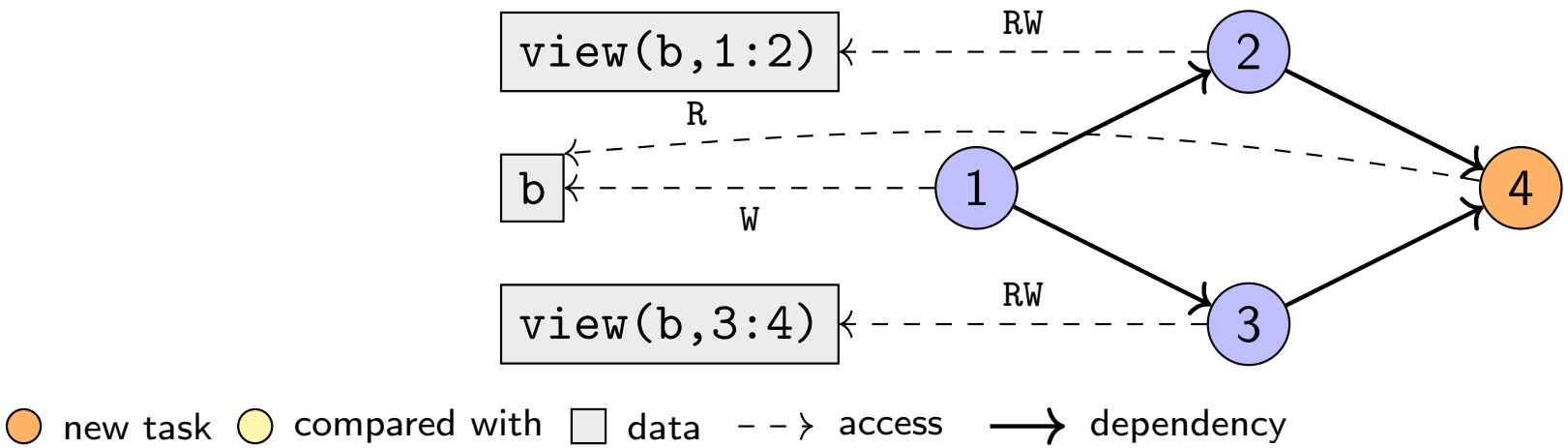
```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
t4 = @dspawn A \ @R(b)
fetch(t4)
```

Behind the scenes: DAG

```
# Note the quadratic complexity!
for i in 1:N, j in i-1:-1:1
  # Detect conflict between i and j
  for di in data(i), dj in data(j)
    if memory_overlap(di, dj)
      add_edge(j, i)
```

Basic idea

1. Extract data dependency from user annotations
2. Infer task dependency from data dependency
3. Schedule tasks with inferred dependencies



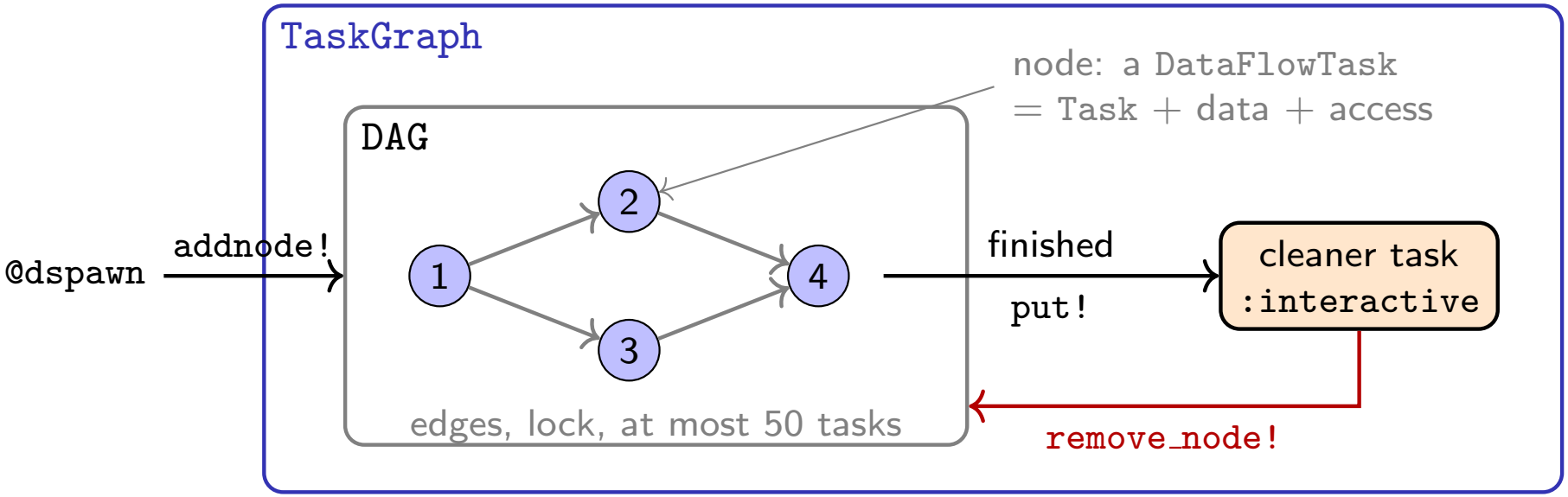
User-written code

```
b = rand(4)
@dspawn fill!(@W(b),0)
@dspawn @RW(view(b,1:2)) .+= 2
@dspawn @RW(view(b,3:4)) .+= 3
t4 = @dspawn A \ @R(b)
fetch(t4)
```

Behind the scenes: scheduling

```
t4 = @spawn begin
  wait(t2); wait(t3)
  A \ b
end
```

Under the hood: main types



- `DataFlowTask`: a `Task` + the data it accesses, and how
- `DAG`: the dependencies between `DataFlowTasks`
- `TaskGraph`: a `DAG` + a task that removes finished tasks

Built from the Part 1 primitives: `Task`, `wait`, `lock`, `Condition`, `Channel`

`@dspawn`: what the macro does

- Scan the expression for `@R`, `@W`, `@RW`; collect `data` and `access_mode`
- Remove the annotations, wrap the code in a closure
- Call the `DataFlowTask` constructor, then schedule it

User-written code

```
@dspawn foo!(@W(B), @R(A)) label = "foo!" priority = 1
```

Macro expansion (approx.)

```
t = DataFlowTask(() -> foo!(B, A), (B, A), (WRITE, READ);
                 label = "foo!", priority = 1)  # 0: most urgent
t.task.sticky = false        # like @spawn
_spawn_set_thrpool(t.task, :default)  # else: parent's pool
schedule(t.task)
```

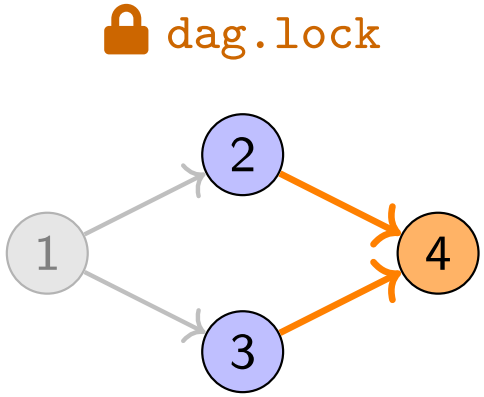
- Returns `t.task`, a plain `Task`: `wait`, `fetch` as usual
- `@dtask`: same, but not scheduled (`schedule` it yourself)

DataFlowTask: a task that waits for its dependencies

A Task, plus the data it accesses, how (access modes), and a priority

```
function DataFlowTask(f, data, mode, prio, tg)
    tj = new(data, mode)  # incomplete: no task yet
    deps = @lock tg.dag.lock begin
        addnode!(tg, tj)
        copy(inneighbors(tg.dag, tj))
    end
    tj.task = @task try
        waitall(deps; failfast = false)
        f()
    finally
        put!(tg.finished, tj)
    end
    tj.task.priority = prio
    return tj
end
```

simplified from src/dataflowtask.jl



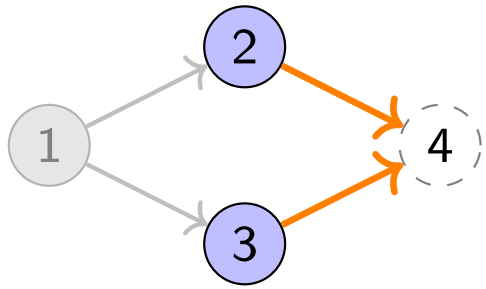
1. **Insert**, under the lock: deps $= \{2, 3\}$, found from the data

DataFlowTask: a task that waits for its dependencies

A Task, plus the data it accesses, how (access modes), and a priority

```
function DataFlowTask(f, data, mode, prio, tg)
    tj = new(data, mode)  # incomplete: no task yet
    deps = @lock tg.dag.lock begin
        addnode!(tg, tj)
        copy(inneighbors(tg.dag, tj))
    end
    tj.task = @task try
        waitall(deps; failfast = false)
        f()
    finally
        put!(tg.finished, tj)
    end
    tj.task.priority = prio
    return tj
end
```

simplified from src/dataflowtask.jl



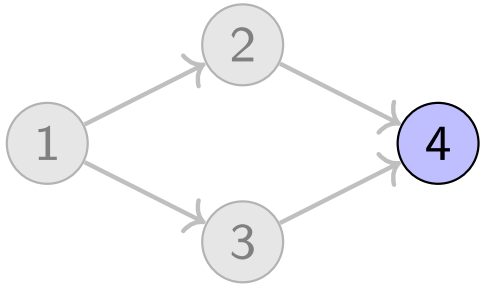
2. Wait for all deps, even if one fails

DataFlowTask: a task that waits for its dependencies

A Task, plus the data it accesses, how (access modes), and a priority

```
function DataFlowTask(f, data, mode, prio, tg)
    tj = new(data, mode)  # incomplete: no task yet
    deps = @lock tg.dag.lock begin
        addnode!(tg, tj)
        copy(inneighbors(tg.dag, tj))
    end
    tj.task = @task try
        waitall(deps; failfast = false)
        f()
    finally
        put!(tg.finished, tj)
    end
    tj.task.priority = prio
    return tj
end
```

simplified from src/dataflowtask.jl



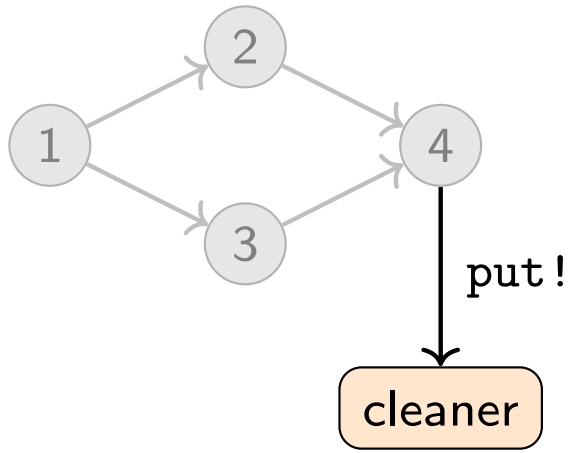
3. Run the code: 2 and 3 are done

DataFlowTask: a task that waits for its dependencies

A Task, plus the data it accesses, how (access modes), and a priority

```
function DataFlowTask(f, data, mode, prio, tg)
    tj = new(data, mode)  # incomplete: no task yet
    deps = @lock tg.dag.lock begin
        addnode!(tg, tj)
        copy(inneighbors(tg.dag, tj))
    end
    tj.task = @task try
        waitall(deps; failfast = false)
        f()
    finally
        put!(tg.finished, tj)
    end
    tj.task.priority = prio
    return tj
end
```

simplified from src/dataflowtask.jl



4. Report in finally: always, even on error; the cleaner removes 4

TaskGraph: where the tasks live

```
mutable struct TaskGraph
    dag::DAG{DataFlowTask}  # tasks + edges
    finished::FinishedChannel
    dag_cleaner::Task
end

# the cleaner, started with the TaskGraph
@spawn :interactive while true
    t = take!(tg.finished)
    t isa Stop && break
    remove_node!(tg, t)
end
```

simplified from `src/taskgraph.jl`

- Holds the unfinished tasks and their edges
- `@dspawn` adds to the active graph: the default one (50 tasks), or `with_taskgraph`
- `wait(tg)`, `resize!(tg, n)`
- Finished task: `put!`; the cleaner (interactive pool): `take!`, `remove_node!`

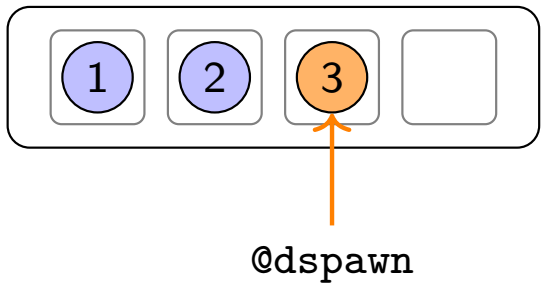
Next: inside the `DAG` →

DAG: a buffered, thread-safe graph

Nodes + edges, one lock, two `Conditions`, `sz_max` slots (default: 50)

```
function addnode!(dag, node)
    @lock dag begin
        while isfull(dag)  # sz_max nodes
            wait(dag.cond_push)
        end
        push!(dag.inoutlist, node => ...)
        update_edges!(dag, node)
    end
end
function remove_node!(dag, node)
    @lock dag begin
        pop!(dag.inoutlist, node)
        notify(dag.cond_push, all=false)
        isempty(dag) &&
            notify(dag.cond_empty)
    end
end
```

simplified from `src/dag.jl`



1. Room: insert, then find the edges

newest to oldest; skip read/read and tasks already reachable

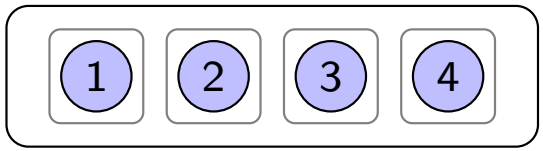
Same pattern as a bounded `Channel`: `put!` waits when full

DAG: a buffered, thread-safe graph

Nodes + edges, one lock, two `Conditions`, `sz_max` slots (default: 50)

```
function addnode!(dag, node)
    @lock dag begin
        while isfull(dag)  # sz_max nodes
            wait(dag.cond_push)
        end
        push!(dag.inoutlist, node => ...)
        update_edges!(dag, node)
    end
end
function remove_node!(dag, node)
    @lock dag begin
        pop!(dag.inoutlist, node)
        notify(dag.cond_push, all=false)
        isempty(dag) &&
            notify(dag.cond_empty)
    end
end
```

simplified from `src/dag.jl`



5 `wait(cond_push)`

2. Full: `@dspawn` waits; `wait` releases the lock

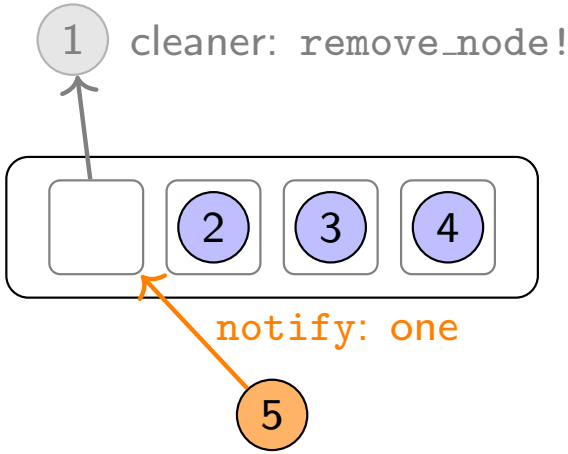
Same pattern as a bounded `Channel`: `put!` waits when full

DAG: a buffered, thread-safe graph

Nodes + edges, one lock, two `Conditions`, `sz_max` slots (default: 50)

```
function addnode!(dag, node)
    @lock dag begin
        while isfull(dag)  # sz_max nodes
            wait(dag.cond_push)
        end
        push!(dag.inoutlist, node => ...)
        update_edges!(dag, node)
    end
end
function remove_node!(dag, node)
    @lock dag begin
        pop!(dag.inoutlist, node)
        notify(dag.cond_push, all=false)
        isempty(dag) &&
            notify(dag.cond_empty)
    end
end
```

simplified from `src/dag.jl`



3. Removal: room for one ⇒ wake one waiter

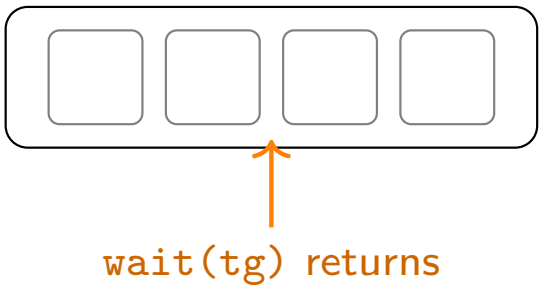
Same pattern as a bounded `Channel`: `put!` waits when full

DAG: a buffered, thread-safe graph

Nodes + edges, one lock, two `Conditions`, `sz_max` slots (default: 50)

```
function addnode!(dag, node)
    @lock dag begin
        while isfull(dag)  # sz_max nodes
            wait(dag.cond_push)
        end
        push!(dag.inoutlist, node => ...)
        update_edges!(dag, node)
    end
end
function remove_node!(dag, node)
    @lock dag begin
        pop!(dag.inoutlist, node)
        notify(dag.cond_push, all=false)
        isempty(dag) &&
            notify(dag.cond_empty)
    end
end
```

simplified from `src/dag.jl`



4. **Empty**: wake `wait(tg)`

Same pattern as a bounded `Channel`: `put!` waits when full

Part 1, inside `DataFlowTasks.jl`

Part 1 primitive	**inside** `DataFlowTasks.jl`
`Task`, `sticky`, pools	`@dspawn` on `:default`; cleaner on `:interactive`
`wait` / `waitall`	a task waits for its dependencies
`lock`	insertion in the `DAG`
`Condition`: `wait`/`notify`	`DAG` full: `@dspawn` waits; empty: `wait(tg)`
`Channel`: `put!`/`take!`	finished tasks $\rightarrow$ the cleaner
atomics	task counter (`tag`)

The package = the Part 1 toolbox, assembled

Tiled LU with `DataFlowTasks.jl`

The sequential loops of Part 1, each kernel annotated:

```
for k in 1:n
    @dspawn lu!(@RW(T[k,k]), NoPivot())
    for j in k+1:n  # panels
        @dspawn ldiv!(UnitLowerTriangular(@R(T[k,k])), @RW(T[k,j]))
        @dspawn rdiv!(@RW(T[j,k]), UpperTriangular(@R(T[k,k])))
    end
    for j in k+1:n, i in k+1:n  # updates
        @dspawn mul!(@RW(T[i,j]), @R(T[i,k]), @R(T[k,j]), -1, 1)
    end
end
A = parent(T[1,1])  # the whole matrix
wait(@dspawn @R(A))  # runs after every write
```

- No `W` table, no barrier: the loop only creates tasks; one `wait`, at the end
- Remove the macros: the sequential code, unchanged

</> Live demo: `lu_dft.jl`

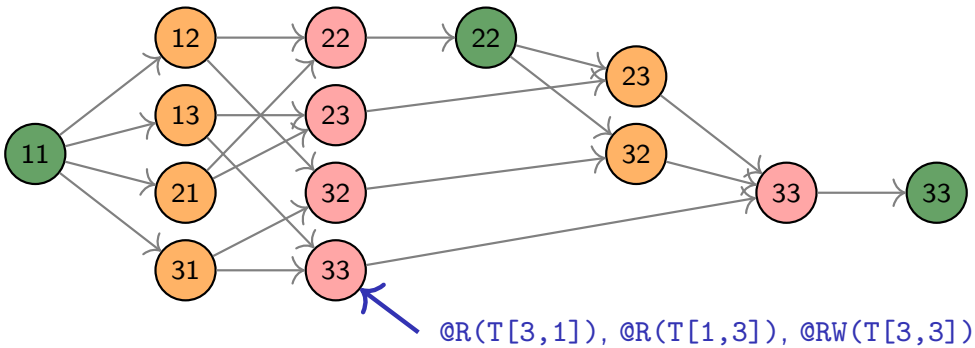
By hand vs. from the data

By hand (Part 1)

```
W[i,j] = spawn_after(W[i,k], W[k,j], W[i,j]) do
    mul!(T[i,j], T[i,k], T[k,j], -1, 1)
end
```

From the data

```
@dspawn mul!(@RW(T[i,j]),
             @R(T[i,k]), @R(T[k,j]), -1, 1)
```



- Same graph: the last writers are found from the annotations
- Plus write-after-read: a tile is not overwritten while others still read it

Profiling: `@log` and `describe`

```
import DataFlowTasks as DFT
log_info = DFT.@log lu_dft!(T)  # 16x16 tiles, 8 threads
DFT.describe(log_info; categories = ["lu", "ldiv", "rdiv", "mul"])
```

```
* Elapsed time                    : 0.176
  +- Longest path                 : 0.063
  +- No-wait (running / workers): 0.148
* Worker time (8 x elapsed)      : 1.410
  +- Running tasks                : 1.188  (84.2%)
  |  +- lu   : 0.040   ldiv : 0.079   rdiv : 0.069   mul : 0.999
  +- Idle                         : 0.222  (15.8%)
* Task insertion                  : 0.010  (5.5% of elapsed)
```

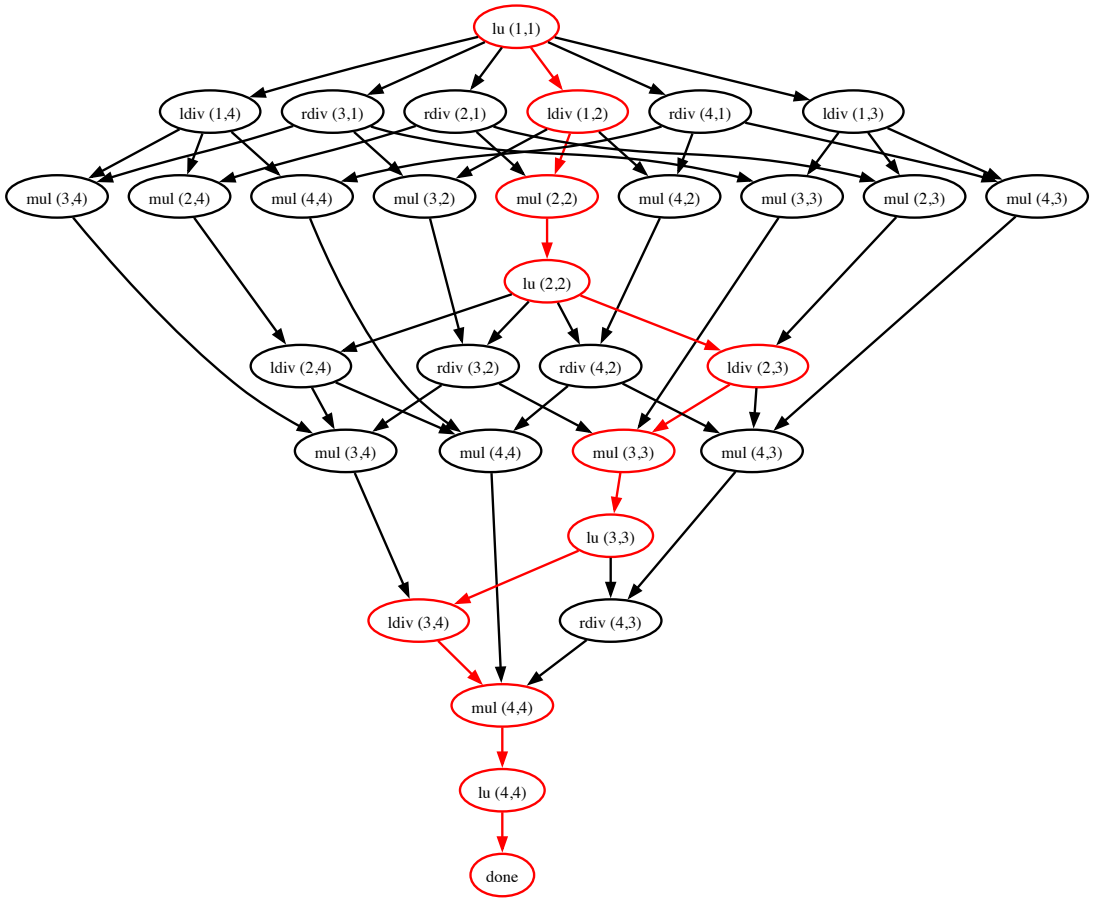
- **Longest path**: lower bound with ∞ threads; **no-wait**: running time / threads
- Elapsed $\approx$ no-wait $\gg$ longest path $\Rightarrow$ work-bound: more threads would help

</> Live demo: `lu_profile.jl`

Profiling: the DAG

```
using GraphViz
GraphViz.Graph(log_info)
```

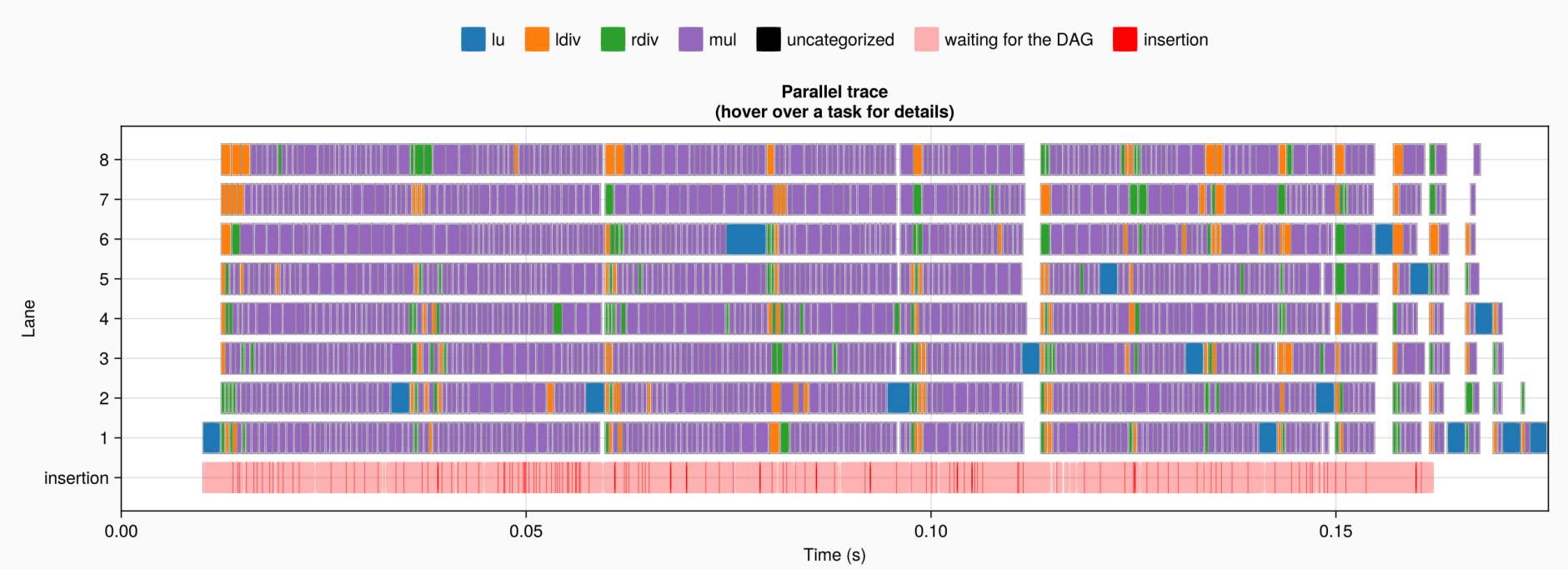
- 4×4 tiles: the graph of the 3×3 picture, one size up
- Red: longest path
- `done`: the final `@R(A)`, after every write
- A cheap check of your annotations



</> Live demo: `lu_profile.jl`

Profiling: the trace

```
using GLMakie  # interactive; CairoMakie: static
plot(log_info; categories = ["lu", "ldiv", "rdiv", "mul"])
```



- One bar per task, colored by category; one lane per thread
- Bottom lane: insertion in the DAG (red), waiting for room (pink)
- Live: hover a task for its dependencies and lifecycle, zoom on the lanes

</> Live demo: `lu_profile.jl`

Tiled LU: results

4000×4000, 8 threads, time in s — Apple M3 Pro, `DataFlowTasks.jl` 0.3

tile size	500	250	200	125	100
tiled (1 thread)	0.92	0.86	0.85	0.93	0.89
fork-join	0.29	0.18	0.17	0.17	0.17
tasks by hand	0.25	0.16	0.15	0.17	0.20
`DataFlowTasks.jl`	0.25	0.16	0.16	0.18	0.23

LAPACK `lu!` (with pivoting), 8 BLAS threads: 0.19 s

- **Fork-join is strong**: `mul!` is 84 % of the work, and independent within a step
- **Tasks win** where barriers idle threads: lone `lu!`, few tiles per thread, many cores
- `DataFlowTasks.jl`: same graph as by hand, derived from the sequential code; small tiles ⇒ insertion cost
- **Tile size**: large ⇒ little parallelism; small ⇒ overhead

Live demo: `lu_table.jl`

Pitfalls

- **Annotations are a contract**: missing or wrong ⇒ data race, no warning
- **Data now, code later**: annotations are evaluated at `@dspawn`, the body when the task runs

```
@dspawn step!(@W(V), @R(U))
U, V = V, U  # the task body may see the swapped names!
```

  ⇒ closures capture variables, not values: `let U = U, V = V`
- **Granularity**: insertion cost $\propto$ tasks in the graph ⇒ tasks of $\gtrsim$ 100 µs
- **Inside a task**: avoid nested threading (`BLAS.set_num_threads(1)`)

Concluding remarks

- `DataFlowTasks.jl` makes parallel programming easier for *certain* algorithms
- Simple API: `@dspawn` + `@R`, `@W`, `@RW`
- **Use it for**: many dependent tasks on shared data (tiles, blocks, ...)
- **Not for**: tiny tasks (insertion cost), or independent loops (`@threads`)

`https://github.com/maltezfaria/DataFlowTasks.jl`

Next: your turn

Table of Contents

Multithreading in Julia
- Tasks and threads
- Independent iterations
- Dependent tasks

DataFlowTasks.jl

Hands-on

Hands-on: your call

Explore the scripts (`scripts/`)

- Part 1: `threads_loop`, `buffers_threads`, `accumulate`, `lu_tiled`, ...
- Part 2: `lu_dft`, `lu_profile`

Try

- More threads, other tile / chunk sizes
- Profile with `DFT.@log`: what limits the run?
- Priorities for the `lu!`s; Cholesky

Bring your own code

- A loop to parallelize, buffers to manage
- A task graph to express with `@dspawn`

Or a project

- DataFrames.jl: threaded `ByRow`
- SparseArrays.jl: threaded `sparse(I,J,V)`
- `@dspawn` on DataFrames pipelines

Ask questions, we are around!

Take-home

- Write tasks, not threads: no `threadid()` bookkeeping
- Buffers belong to tasks: one per chunk, or borrowed from a pool
- Shared mutable state: avoid it (partial sums), or protect it (lock, atomic)
- Dependent tasks: express the dependencies, ideally from the data (`DataFlowTasks.jl`)

- Material: https://github.com/maltezfaria/2026-frejus
- `DataFlowTasks.jl` docs: github.com/maltezfaria/DataFlowTasks.jl
- Julia manual: *Multi-Threading*, *Asynchronous Programming*

Thank you!

Backup slides

Threads vs. cores: why the OS layer?

- **Thread** = software (stack + registers); **core** = hardware
  - OS can pause a thread, save its state, resume it on any core
- Many more threads than cores ($\sim$1000s vs. $\sim$10)
  - OS time-slices and preempts threads
  - programs need not know the number of cores
- **Blocking is cheap**: thread waits on I/O $\Rightarrow$ OS runs another one
- **Only the kernel schedules onto cores**
  - programs can only *request* an affinity
  - HPC: pin threads to cores (cache, NUMA)
    `ThreadPinning.jl`, `JULIA_EXCLUSIVE=1`
- Hyperthreading (SMT): 1 physical core = 2 hardware threads

Why tasks on top of threads?

	OS thread	**Julia task**
Scheduled by	kernel	Julia runtime (user space)
Memory	∼MB stack	∼KB
Switch cost	kernel call	cheap (user space)
Interruption	preemptive	cooperative (yield points)
Reasonable number	∼#cores	10^3–10^6

- **M:N scheduling**: M tasks on N threads (N ≈ #cores)
- Nested `@spawn` ⇒ more tasks, not more threads
- Task switches only at `wait`, `sleep`, I/O, `yield`, ...
  - a busy loop keeps its thread

Fix 2: atomic

```
ld = Atomic{Float64}(0.0)
@threads for i in 1:N
    F = lu(As[i])
    xs[i] = F \ bs[i]
    atomic_add!(ld, logdet(F))
end
ld[]  # read the value
```

- One indivisible read-add-write: no lock object, no `Ref`
- Lower level: a hardware operation, not a task-level lock
- Only primitive types: `Int`, `Float64`, ...
- Correct, same speed: 0.47 s (`-t 4`)
- Lock or atomic: order of additions varies ⇒ last digit varies

</> Live demo: `accumulate.jl`

Also fine: partial sums

```
ts = map(partition(1:N, n)) do idxs
    @spawn begin
        s = 0.0
        for i in idxs
            F = lu(As[i])
            xs[i] = F \ bs[i]
            s += logdet(F)
        end
        s  # returned by the task
    end
end
ld = sum(fetch.(ts))
```

- The chunk pattern again: no shared state, `fetch` returns it
- Same order every run ⇒ same digits

</> Live demo: `accumulate.jl`

Example: longest common subsequence (LCS)

Subsequence: same order, gaps allowed `diff`, DNA alignment, ...

$x =$ `GAC`, $y =$ `AGCAT` $\Rightarrow$ LCS $=$ `GA` or `AC`, length 2

	∅	A	G	C	A	T
∅	0	0	0	0	0	0
G	0	0	1	1	1	1
A	0	1	1	1	2	2
C	0	1	1	2	2	2

C = C $\Rightarrow$ diagonal $+1$

$L_{ij} =$ LCS length of $x_{1:i}$ and $y_{1:j}$:

$$L_{ij} = \begin{cases} L_{i-1,j-1} + 1 & x_i = y_j \\ \max(L_{i-1,j},\, L_{i,j-1}) & \text{otherwise} \end{cases}$$

- First row and column: 0
- Answer: $L_{\text{end,end}}$, then backtrack
- Cost: $O(|x|\,|y|)$

LCS: why is it hard to parallelize?

1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8
5	6	7	8	9

tiles, numbered by step: a wavefront

- Cell needs top, left, diagonal: rows not independent, `@threads` ✗
- Anti-diagonals are, but a cell is one comparison
- Tiles: (i, j) needs $(i-1, j)$ and $(i, j-1)$

By hand (Part 1):

```
T = Matrix{Task}(undef, n, n)
for j in 1:n, i in 1:n
    T[i,j] = @spawn begin
        i > 1 && wait(T[i-1,j])
        j > 1 && wait(T[i,j-1])
        fill_lengths!(L, x, y, I[i], J[j])
    end
end
```

- Works, but you derived the graph: indices, borders, init, ...

LCS with `DataFlowTasks.jl`

Tiled, sequential

```
L[:, 1] .= 0
L[1, :] .= 0
for jr in J, ir in I

    fill_lengths!(L, x, y, ir, jr)


end
backtrack(L, x, y)
```

Parallel

```
@dspawn fill!(@W(view(L, :, 1)), 0)
@dspawn fill!(@W(view(L, 1, :)), 0)
for jr in J, ir in I
    @dspawn begin
        @R view(L, ir, jr)
        @RW view(L, ir .+ 1, jr .+ 1)
        fill_lengths!(L, x, y, ir, jr)
    end
end
fetch(@dspawn backtrack(@R(L), x, y))
```

- Same loops (`I`, `J`: tiles); each tile says what it reads and writes
- Tile reads its top/left neighbors (overlap with `ir`, `jr`), writes itself
- The wavefront emerges from the annotations

LCS: performance

x, y of length 8192 and 16384; 16×16 tiles

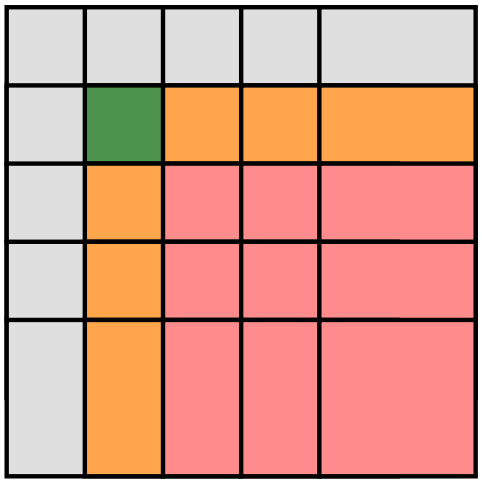
	`-t 1`	`-t 4`	`-t 8`
sequential	0.10 s	0.10 s	0.10 s
tasks by hand (`wait`)	0.12 s	0.042 s	0.029 s
`DataFlowTasks.jl`	0.12 s	0.043 s	0.033 s

- Same speed as by hand, without deriving the graph
- $\approx 3\times$ on 8 threads: memory-bound, and the wavefront must ramp up
- Too many tiles (64×64): slower, task overhead dominates

</> Live demo: `lcs.jl`

Tiled LU: the algorithm

The 2×2 block LU of Part 1, applied tile by tile:



step $k = 2$ of 5

`for k in 1:n`

1. diagonal: $T_{kk} \leftarrow L_{kk} U_{kk}$ `lu!`
2. row panel: $T_{kj} \leftarrow L_{kk}^{-1} T_{kj}$ `ldiv!`

   column panel: $T_{ik} \leftarrow T_{ik} U_{kk}^{-1}$ `rdiv!`
3. trailing update: $T_{ij} \leftarrow T_{ij} - T_{ik} T_{kj}$ `mul!`

- $O(n)$ `lu!`, $O(n^2)$ `ldiv!`/`rdiv!`, $O(n^3)$ `mul!`: compute-bound
- Gray: done. In place: L and U overwrite A